

1.First-Come,First-Served CPU Scheduling Algorithm

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Completed

First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $SWT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5
6     printf("Enter number of processes: \n");
7     scanf("%d", &n);
8
9     int at[100], bt[100], ct[100], tat[100], wt[100];
10
11     for (int i = 0; i < n; i++) {
12         printf("Enter Arrival Time and Burst Time for P%d: \n", i + 1);
13         scanf("%d %d", &at[i], &bt[i]);
14     }
15
16     ct[0] = at[0] + bt[0];
17
18     for (int i = 1; i < n; i++) {
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Completed

First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $SWT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.

Select Language: C (GCC 9.2.0)

```
20         ct[i] = at[i] + bt[i];
21     } else {
22         ct[i] = ct[i - 1] + bt[i];
23     }
24 }
25
26 float total_tat = 0, total_wt = 0;
27
28 for (int i = 0; i < n; i++) {
29     tat[i] = ct[i] - at[i];
30     wt[i] = tat[i] - bt[i];
31
32     total_tat += tat[i];
33     total_wt += wt[i];
34 }
35
36 float avg_tat = total_tat / n;
37 float avg_wt = total_wt / n;
38
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Back To Practice

< Q1 >

Save

First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come,First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.

Select Language: C (GCC 9.2.0)

```
26 float total_tat = 0, total_wt = 0;
27
28 for (int i = 0; i < n; i++) {
29     tat[i] = ct[i] - at[i];
30     wt[i] = tat[i] - bt[i];
31
32     total_tat += tat[i];
33     total_wt += wt[i];
34 }
35
36 float avg_tat = total_tat / n;
37 float avg_wt = total_wt / n;
38
39 printf("\nAverage Turnaround Time: %.2f", avg_tat);
40 printf("\nAverage Waiting Time: %.2f\n", avg_wt);
41
42 return 0;
43 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1